

Time and Space

Q1:

```
for (int i = 0; i < n; i += 2) {  
    printf("%d ", i);  
}
```

- **Hint:** Linear loop with constant increment.
- **Time Complexity:** ?
- **Space Complexity:** ?

Q2:

```
for (int i = n; i > 0; i = i / 2) {  
    printf("%d ", i);  
}
```

- **Hint:** Divide-by-constant loop
- **Time Complexity:** ?
- **Space Complexity:** ?

Q3:

```
void fun(int n) {  
    if (n == 0) return;  
    fun(n / 2);  
    printf("%d ", n);  
}
```

- **Hint:** One recursive call per halving
- **Time Complexity:** ?
- **Space Complexity:** ?

Q4:

```
int sum(int n) {  
    if (n == 0) return 0;  
    return n + sum(n - 1);  
}
```

- **Hint:** n recursive calls
- **Time Complexity:** ?
- **Space Complexity:** ?

Q5:

```
void expFunc(int n) {
```

```

    if (n == 0) return;
    expFunc(n - 1);
    expFunc(n - 1);
}

```

- **Hint:** Two calls per level → binary tree
- **Time Complexity:** ?
- **Space Complexity:** ?

Q6:

```

for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j = j * 2) {
        printf("%d %d\n", i, j);
    }
}

```

- **Hint:** Outer = n, Inner = log n
- **Time Complexity:** ?
- **Space Complexity:** ?

Q7:

```

for (int i = 0; i < n; i++) {
    if (i % 2 == 0)
        printf("%d ", i);
}

```

- **Hint:** Runs n times, condition doesn't affect complexity
- **Time Complexity:** ?
- **Space Complexity:** ?

Q8:

```

for (int i = 0; i < n; i++) {
    int a = i * i;
    printf("%d ", a);
}

```

- **Hint:** One small variable per loop
- **Time Complexity:** ?
- **Space Complexity:** ?

Q9:

```

for (int i = 0; i < n; i++) {
    printf("Linear %d\n", i);
}
for (int i = 1; i < n; i *= 2) {
    printf("Log %d\n", i);
}

```

```
}  
int a = 5 + 3;
```

- **Time Complexity: ?**
- **Space Complexity: ?**

Q10:

```
void loopThenRecurse(int n) {  
    for (int i = 0; i < 10; i++)  
        printf("%d ", i);  
    if (n == 0) return;  
    loopThenRecurse(n - 1);  
}
```

- **Hint:** Constant loop, n recursive calls
- **Time Complexity: ?**
- **Space Complexity: ?**